



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 03

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

N° de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 12/09/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para realizar el ejercicio de clase, fue necesario usar 14 figuras, 8 pirámides triangulares y 6 cubos que siguen el patrón de colores que utilicé en mi práctica anterior, primero definí los cubos color azul, rotados 45 grados en el eje Z, para posteriormente instanciar los cubos café, a la mitad de escala que los azules y los coloqué en las mismas posiciones, son 3 para que unos se muestren enfrente y atrás, otros arriba y abajo y otros izquierda y derecha, así se cubren todas las caras, los cubos café son 0.01 más anchos en escala que los azules para que sobresalga el cuadro café.

```
// =====  
// PRIMER CUBO (Azul Z)  
// =====  
model = baseModel; // Partimos de la matriz base  
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f)); // Posición del primer cubo  
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.41f));  
model = glm::rotate(model, 45.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));  
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));  
color = glm::vec3(0.0f, 0.8f, 0.8f); // Verde  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
meshList[0]->RenderMeshGeometry(); // Dibuja el primer cubo
```

```
// =====  
// SEGUNDO CUBO (Azul X)  
// =====  
model = baseModel;  
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));  
model = glm::scale(model, glm::vec3(1.41f, 1.0f, 1.0f));  
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));  
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));  
color = glm::vec3(0.0f, 0.8f, 0.8f); // Azul  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
meshList[0]->RenderMeshGeometry(); // Dibuja el segundo cubo
```

```
// =====  
// TERCER CUBO (Azul Y)  
// =====  
model = baseModel;  
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));  
model = glm::scale(model, glm::vec3(1.0f, 1.41f, 1.0f));  
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));  
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));  
color = glm::vec3(0.0f, 0.8f, 0.8f); // Azul  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
meshList[0]->RenderMeshGeometry(); // Dibuja el tercer cubo
```

```
// =====
// PRIMER CUBO (Cafe Z)
// =====
model = baseModel; // Partimos de la matriz base
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f)); // Posición del primer cubo
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 1.42f));
model = glm::rotate(model, 45.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.478f, 0.255f, 0.067f); // Café
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry(); // Dibuja el primer cubo
```

```
// =====
// SEGUNDO CUBO (Cafe X)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.42f, 0.5f, 0.5f));
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.478f, 0.255f, 0.067f); // Café
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry(); // Dibuja el segundo cubo
```

```
// =====
// TERCER CUBO (Café Y)
// =====
model = baseModel; // Reiniciamos a baseModel para que no dependa del primer cubo
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.47f, 0.5f));
model = glm::rotate(model, -45.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.478f, 0.255f, 0.067f); // Café
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry(); // Dibuja el tercer cubo
```

Posteriormente, usamos la figura de pirámide triangular para realizar los vértices de colores de la figura, fueron 8 en total y se utilizaron en rojo, amarillo, verde y naranja, orientandose a 0.71 de distancia del origen en todos los puntos para cubrir todas las esquinas del cubo, cambiando su posición solo en negativo o positivo en cada uno de los ejes X, Y o Z. Quedando de la siguiente manera:

```
// =====
// PIRÁMIDE TRIANGULAR 1 (Verde)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.71f, -0.71f, 0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 1.0f, 0.0f); // Verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// =====
// PIRÁMIDE TRIANGULAR 2 (Rojo)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// =====
// PIRÁMIDE TRIANGULAR 3 (Amarillo)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(-0.71f, 0.71f, 0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// =====
// PIRÁMIDE TRIANGULAR 4 (Naranja)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(-0.71f, -0.71f, 0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::rotate(model, 270.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.5f, 0.0f); // Naranja
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

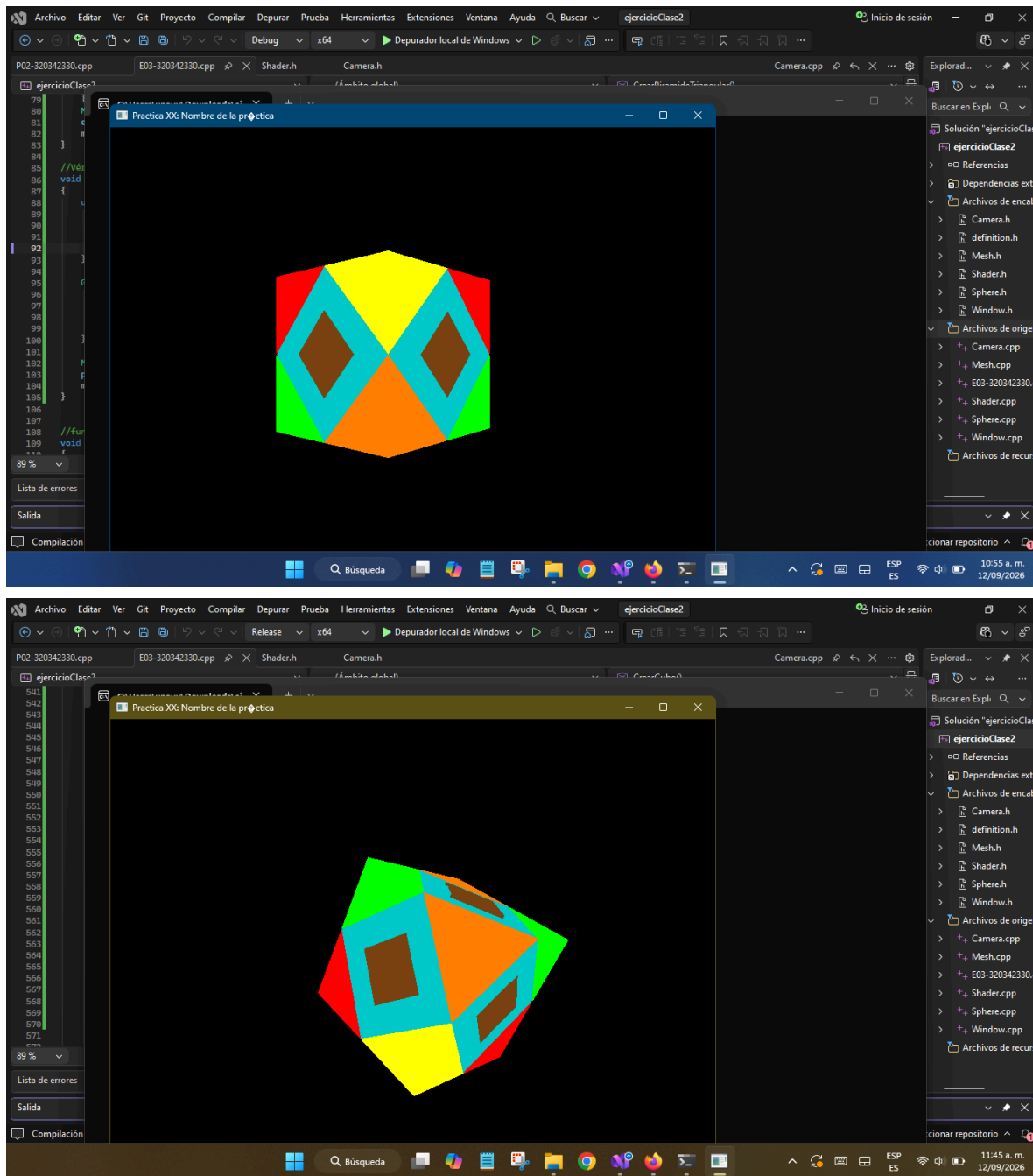
```
// =====
// PIRÁMIDE TRIANGULAR 5 (Verde)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(-0.71f, -0.71f, -0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 1.0f, 0.0f); // Verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// =====
// PIRÁMIDE TRIANGULAR 6 (Rojo)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(-0.71f, 0.71f, -0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// =====
// PIRÁMIDE TRIANGULAR 7 (Naranja)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.71f, -0.71f, -0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.5f, 0.0f); // Naranja
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// =====
// PIRÁMIDE TRIANGULAR 8 (Amarillo)
// =====
model = baseModel;
model = glm::translate(model, glm::vec3(-0.4f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.71f, 0.71f, -0.71f));
model = glm::scale(model, glm::vec3(0.71f, 0.71f, 0.71f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

La ejecución se muestra a continuación:



2. Problemas presentados.

Únicamente tuve problemas con el uso de las pirámides triangulares, pues me fue difícil utilizar las previamente establecidas en el documento, por lo que opté por crear nuevos vértices a la figura para que me fuese más sencillo incluirlas en la figura.

```

//Vértices de pirámide triangular
void CrearPiramideTriangular()
{
    unsigned int indices_piramide_triangular[] = {
        0, 1, 2,
        0, 2, 3,
        0, 3, 1,
        1, 3, 2
    };

    GLfloat vertices_piramide_triangular[] = {
        0.0f, 0.0f, 0.0f,
        0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, -1.0f,
        -1.0f, 0.0f, 0.0f
    };

    Mesh* piramidet = new Mesh();
    piramidet->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
    meshList.push_back(piramidet);
}

```

3. Conclusión:

Con esta práctica comprendí de mejor manera el uso del eje Z para indicar profundidad para crear una figura tridimensional, además de ver el uso de una cámara dinámica que puede moverse con las teclas, me pareció muy buena y considero que la explicación fue muy completa y muy útil para poder realizar el ejercicio.